

5

Lab

PHỤC VỤ MỤC ĐÍCH GIÁO DỤC
FOR EDUCATIONAL PURPOSE ONLY

Hard Disk Forensics

Điều tra bộ nhớ lưu trữ:

Tìm bằng chứng phạm tội trong ổ cứng máy tính

Thực hành môn Pháp chứng kỹ thuật số



Lưu hành nội bộ

<Nghiêm cấm đăng tải trên internet dưới mọi hình thức>

A. TỔNG QUAN

1. Mục tiêu

- Hiểu nguyên tắc và quy trình phân tích pháp chứng ổ đĩa cứng.
- Thực hành thu thập, trích xuất và phân tích dữ liệu từ ổ đĩa.
- Áp dụng các công cụ pháp chứng để xác định dấu vết kỹ thuật số.

2. Thời gian thực hành

- Thực hành tại lớp: 5 tiết tại phòng thực hành.
- Hoàn thành báo cáo kết quả thực hành: tối đa 14 ngày.

3. Môi trường & cấu hình

- Sử dụng các thiết bị và tài liệu, khuyến cáo được cung cấp bởi GVTH, yêu cầu tác phong nghiêm túc trong quá trình thực hiện.
- Công cụ gợi ý: **FTK Imager, Autopsy/Sleuth Kit, TestDisk, ...**
- Tài liệu nên đọc:
 - Sách “**Computer Forensics with FTK**” (tác giả: Fernando Carbone)
 - Sách “**File system forensic analysis**” (tác giả: Brian Carrier)

B. THỰC HÀNH

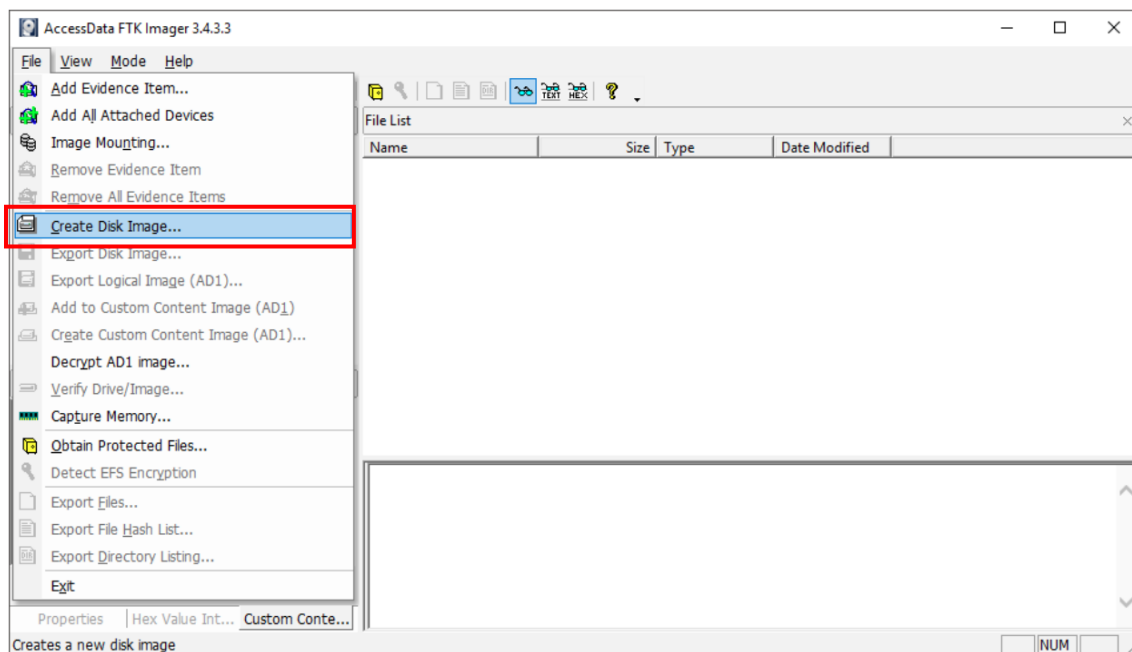
Sinh viên thực hiện điều tra theo yêu cầu của GVHD, làm theo nhóm thực hành đã đăng ký trên lớp trong buổi thực hành.

B1. Thu thập dữ liệu từ ổ đĩa

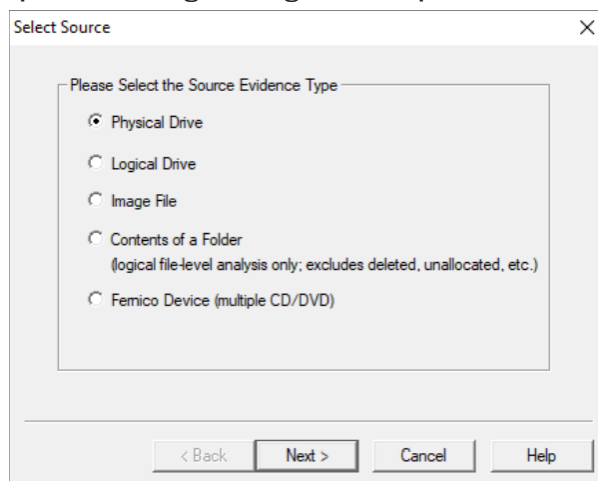
a) Tạo bản sao (forensic image) bằng FTK Imager

FTK Imager là một công cụ dùng để quan sát dữ liệu của chứng cứ số với mục đích có thể quan sát phân tích sâu hơn. **FTK Imager** còn có thể tạo ra một bản sao chi tiết (còn được gọi là forensic images) của máy tính mục tiêu mà hoàn toàn không thay đổi cấu trúc của bản gốc.

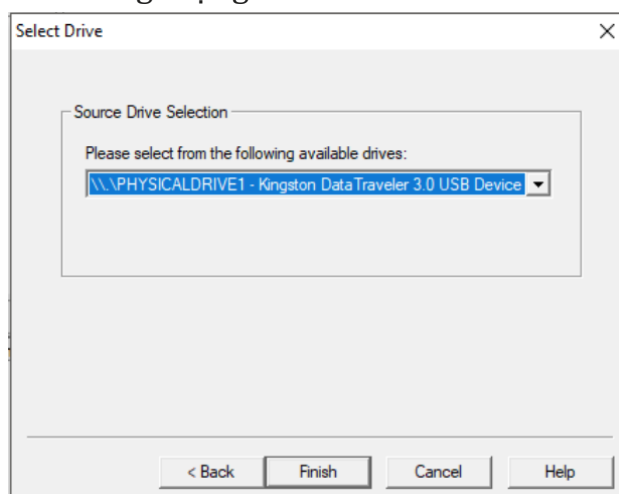
- ❖ Cài đặt công cụ **FTK Imager**, tải về tại: <https://www.exterro.com/ftk-product-downloads/ftk-imager-4-7-3-81>
- ❖ Chức năng tạo RAW Image:
 - **Bước 1:** Chọn *File => Create Disk Image*.



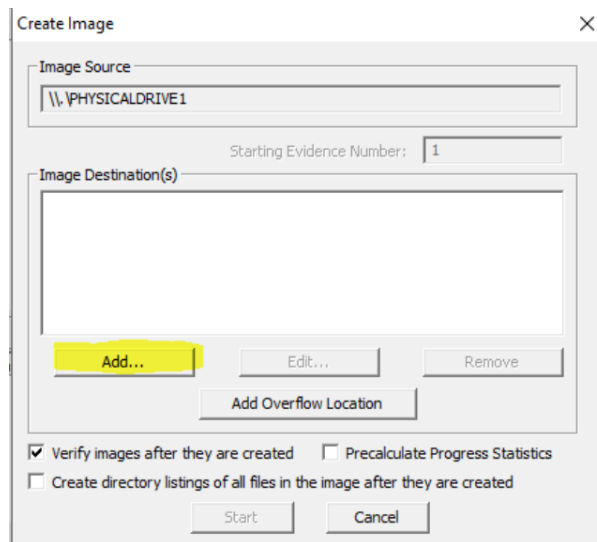
- **Bước 2:** Chọn loại ổ đĩa bằng chứng muốn tạo ra:



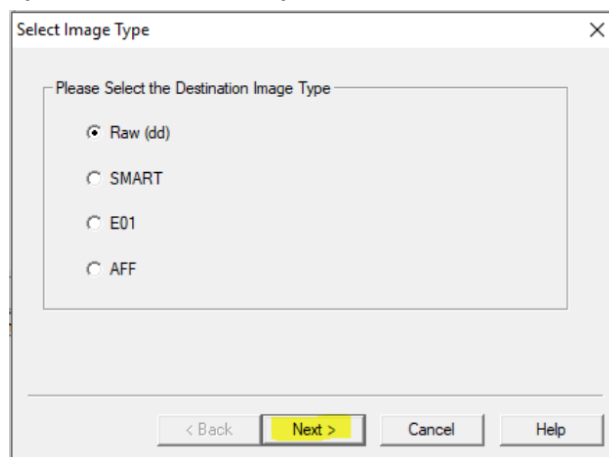
- **Bước 3:** Do dung lượng ổ đĩa của máy tính quá lớn nên trong ví dụ này, chúng ta chọn tạo từ USB với dung lượng vừa đủ.



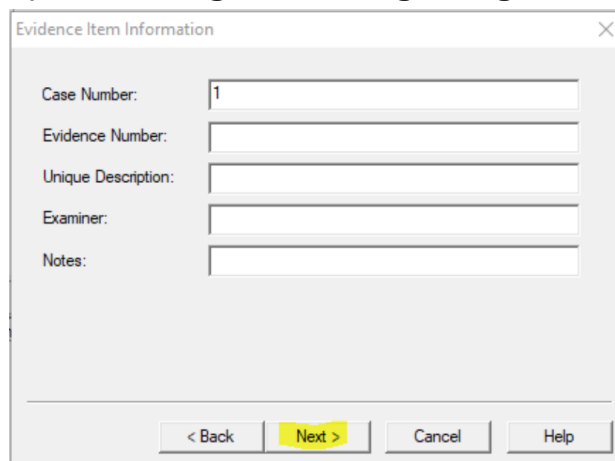
- **Bước 4:** Chọn nơi để lưu file ảnh đĩa



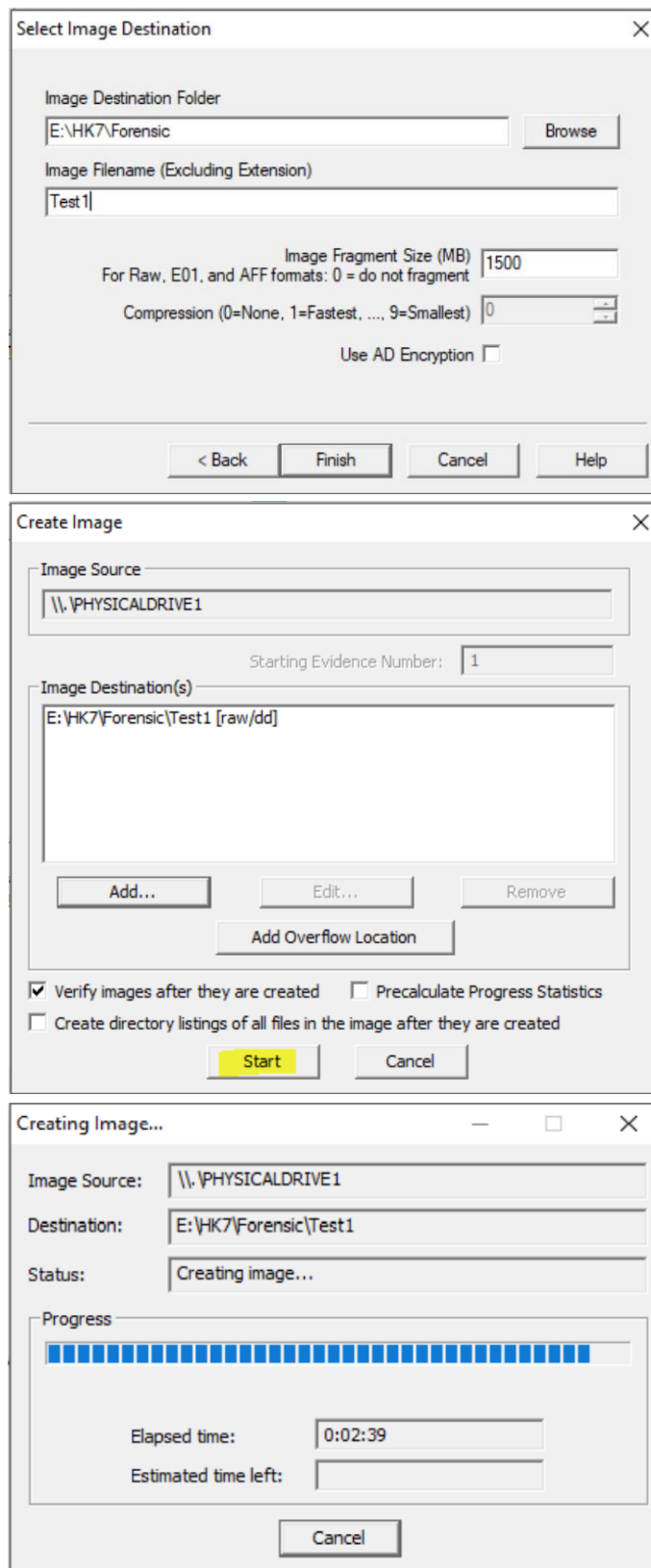
- **Bước 5:** Chọn loại ảnh đĩa muốn tạo ra.



- **Bước 6:** Thực hiện điền thông tin của bằng chứng

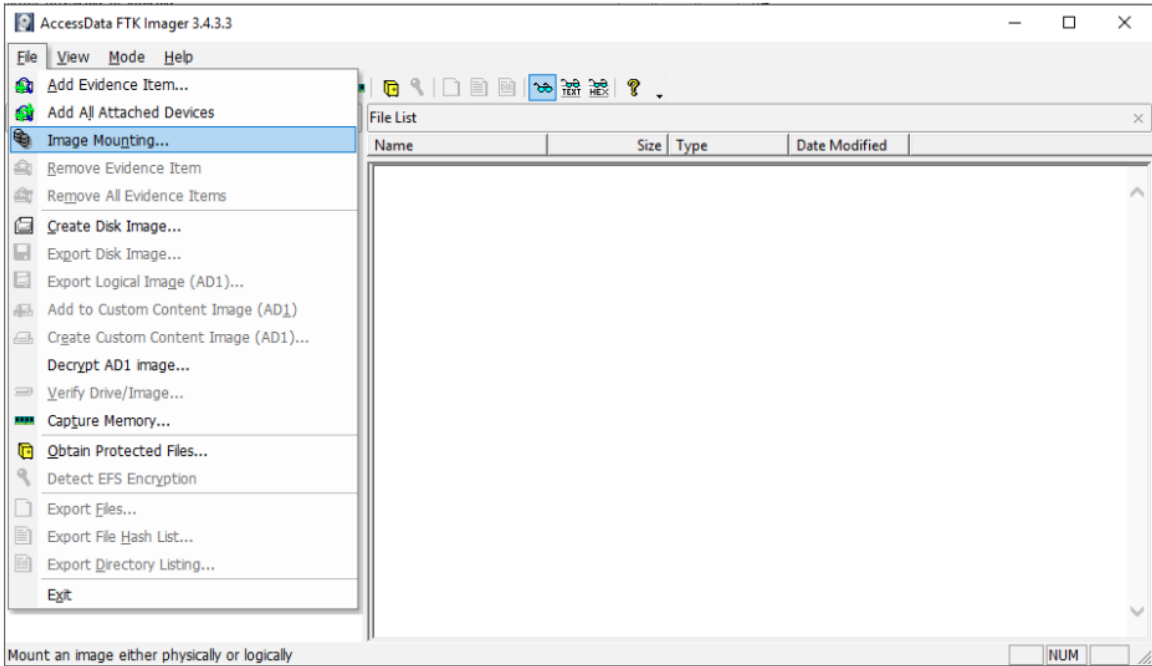


- **Bước 7:** Thực hiện các bước như đặt tên sau đó chọn Start để bắt đầu tạo ảnh đĩa.

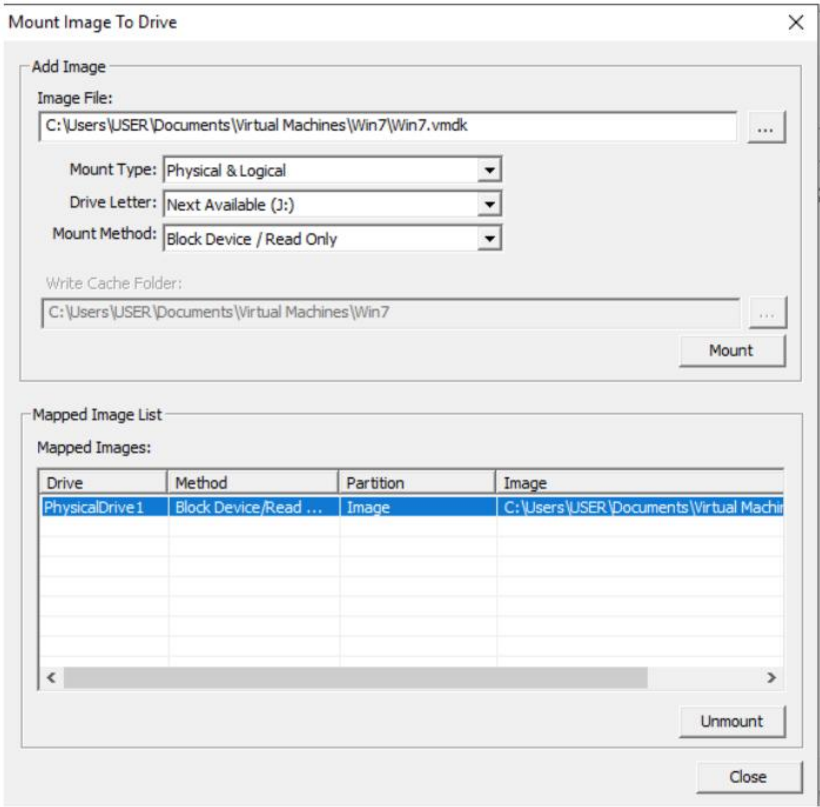


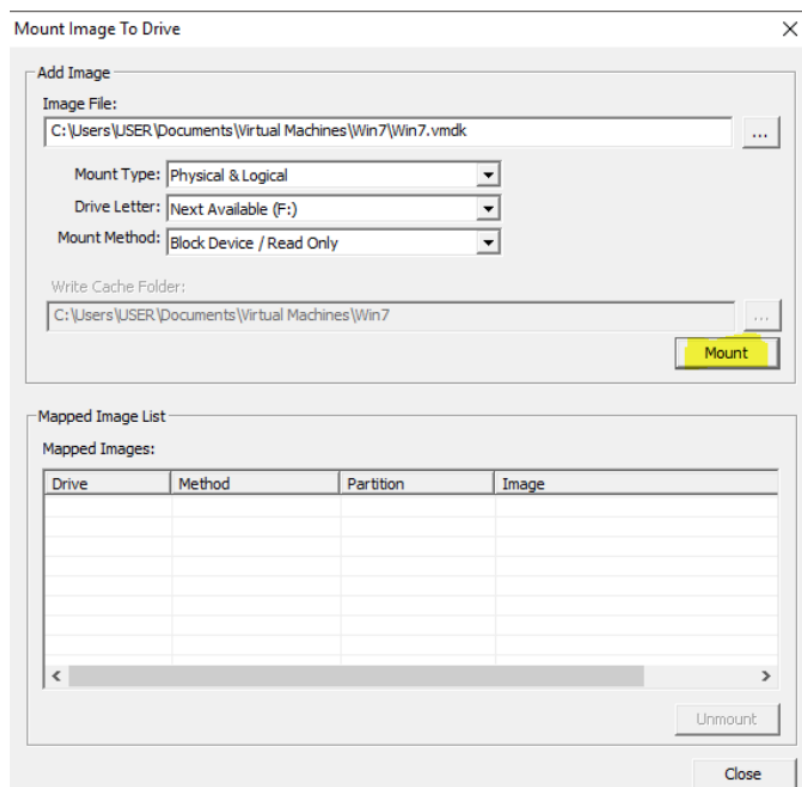
- ❖ Đối với máy ảo, trước khi tạo bản sao ổ đĩa như đã trình bày ở trên, cần thực hiện các bước sau:

Bước 1: Gắn (mounting) file ảnh của ổ đĩa (disk images) vào máy tính phân tích:



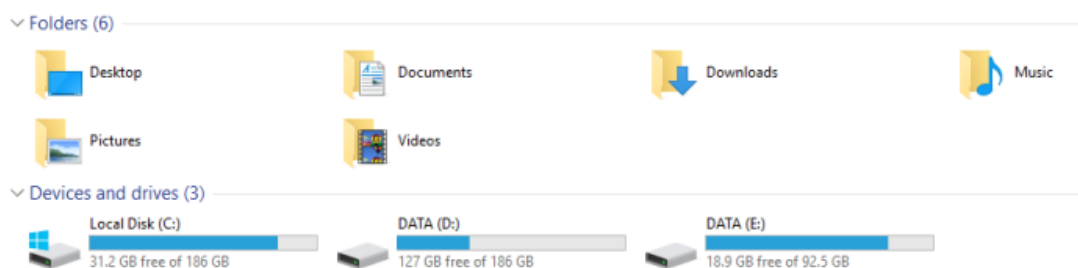
Bước 2: Chọn ảnh đĩa (disk image) mục đích cần phân tích, sau đó chọn Mount để gắn thêm ổ đĩa:



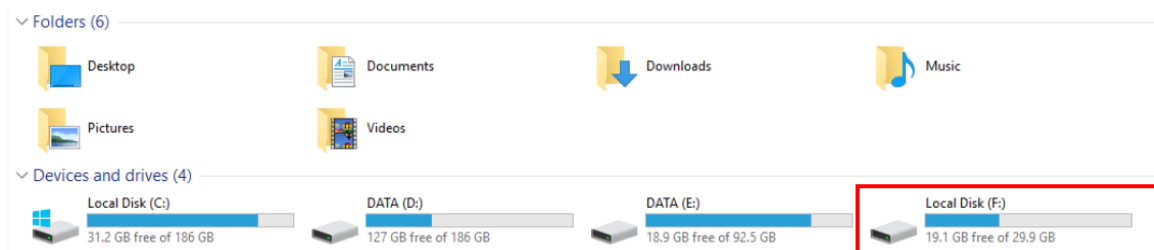


Như vậy, chúng ta đã thêm vào một ổ đĩa với dữ liệu từ file đĩa từ máy ảo Windows 7 để phân tích.

Trước khi thêm:

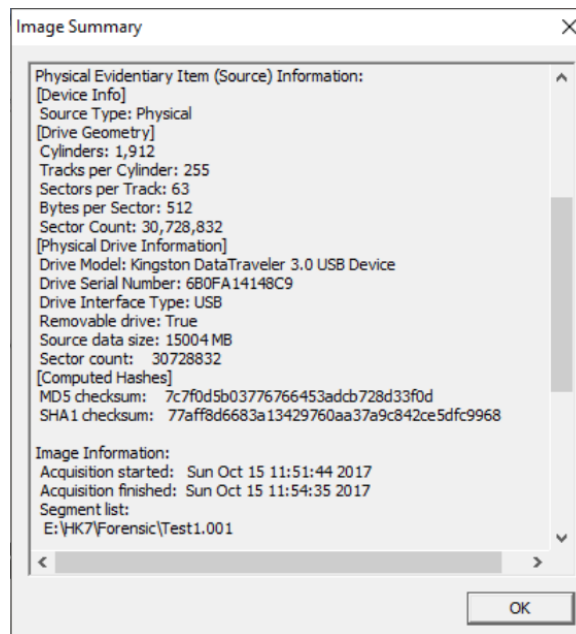


Sau khi gắn thêm file đĩa cần phân tích:

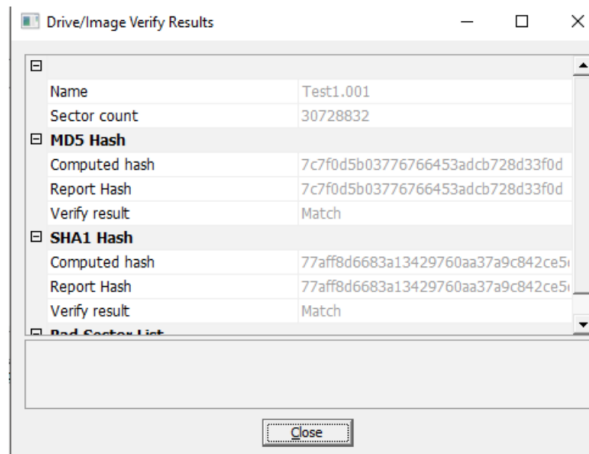


b) Kiểm tra hash để đảm bảo tính toàn vẹn

Bảng tóm tắt của ảnh đĩa vừa được tạo xong như sau:



Kết quả, thông tin của ảnh đĩa sau khi tạo như sau:



B2. Phân tích bộ nhớ lưu trữ

a) Phân tích dữ liệu bằng công cụ Autopsy

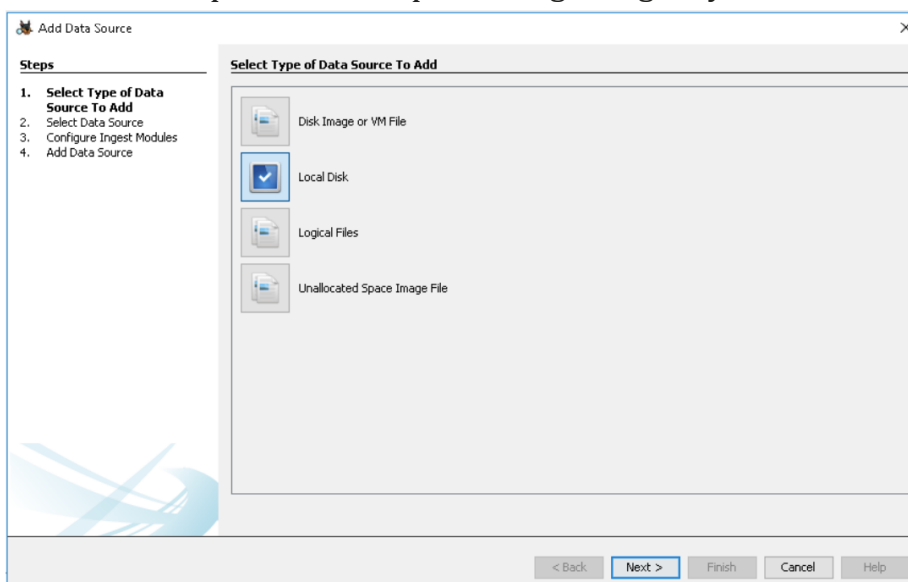
Giúp sinh viên nắm bắt và hiểu rõ các tính năng của công cụ phần mềm Autopsy khi tiến hành điều tra và tìm kiếm thông tin trong một Filesystem.

Autopsy là một công cụ phần mềm pháp chứng kỹ thuật số với giao diện đồ họa của bộ phần mềm mã nguồn mở Sleuth Kit, Autopsy có thể được sử dụng để điều tra những gì đã xảy ra trên máy tính. Autopsy cho phép phân tích sự kiện theo thời gian, trình bày các sự kiện truy cập tới File system theo trình tự với giao diện đồ họa.

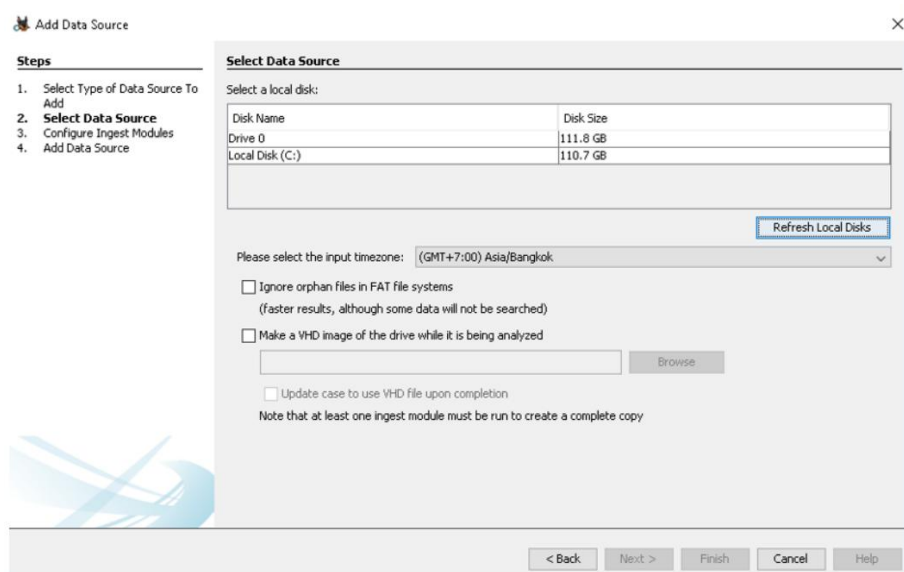
- ❖ Link download: <https://www.sleuthkit.org/autopsy/download.php>
- Khởi động Autopsy để tạo một Case mới, sử dụng lựa chọn "Create New Case".



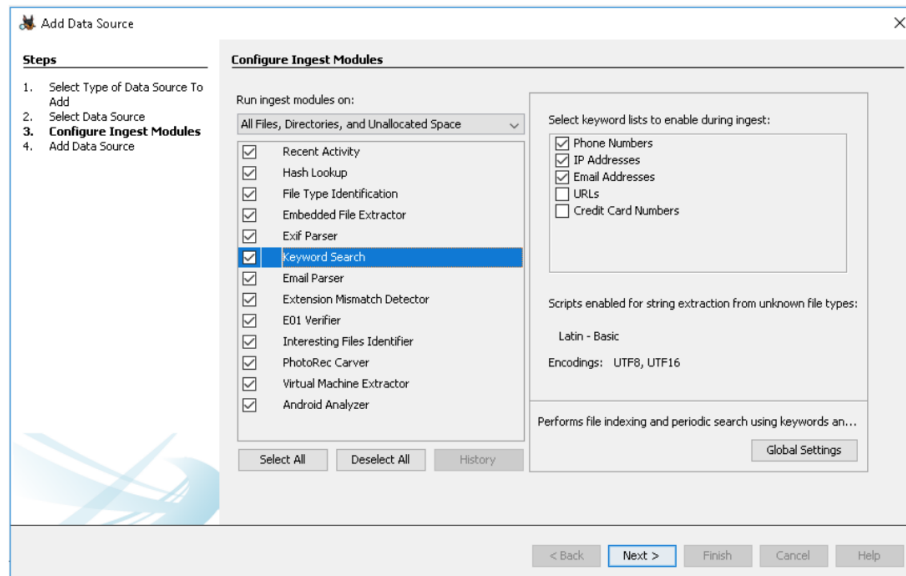
- Điền tên Case vào khung Case name (cũng là tên của thư mục chứa các file liên quan).
- Chọn Local Disk để phân tích các phân vùng trong máy.



- Chọn Disk Name cần phân tích.

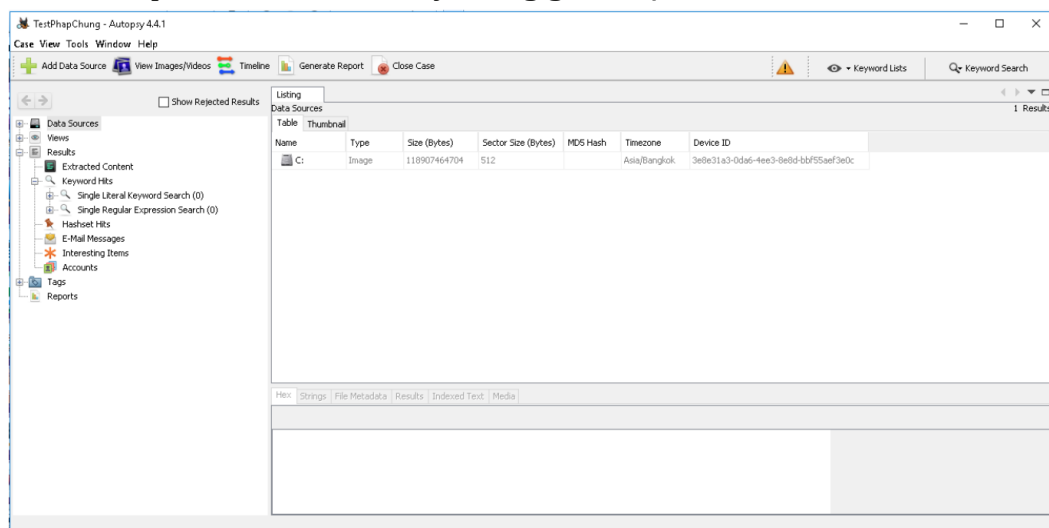


- Chọn ra mô-đun để phân tích, mô-đun Keyword Search sẽ có thêm một số tùy chọn như: IP, email,...



Một số mô-đun trong công cụ:

- **Recent Activity:** Tìm kiếm các hoạt động người dùng như lưu bởi các trình duyệt web và hệ điều hành Windows.
- **Hash Lookup:** Sử dụng cơ sở dữ liệu băm để bỏ qua các tập tin được biết từ NIST NSRL và cò để tìm các tập tin xấu. Sử dụng nút "Advanced" để thêm và cấu hình cơ sở dữ liệu để sử dụng hash trong quá trình này.
- **Keyword Search:** Sử dụng danh sách từ khoá để xác định các tập tin với những từ cụ thể trong đó. Chúng ta có thể chọn danh sách từ khóa để tìm kiếm tự động và tạo danh sách mới bằng cách sử dụng nút "Advanced".
- **Archive Extractor:** Mở các file có dạng ZIP, RAR, và các định dạng lưu trữ khác và tìm các tập tin từ các tập tin lưu trữ để phân tích thông tin.
- **Exif Parser:** Trích xuất thông tin EXIF từ các tập tin hình ảnh là lưu các kết quả hình ảnh vào cây trong giao diện chính.



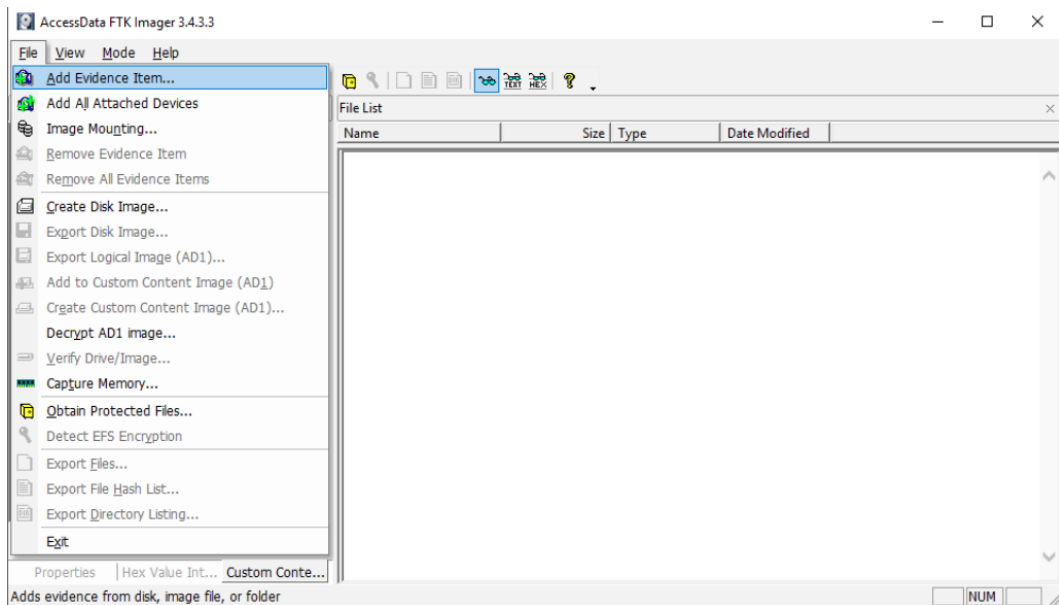
- **Data Sources:** Hiển thị tất cả dữ liệu trong Filesystem trong đó có cấu trúc hệ thống tập tin của hình ảnh đĩa hoặc đĩa cục bộ.
- **Views:** Hiển thị các thông tin chi tiết thông tin của các file chứa trong Filesystem.
- **Result:** Hiển thị và phân loại các thông tin mà các mô-đun phân tích được trong Filesystem.

Bài tập (yêu cầu làm)

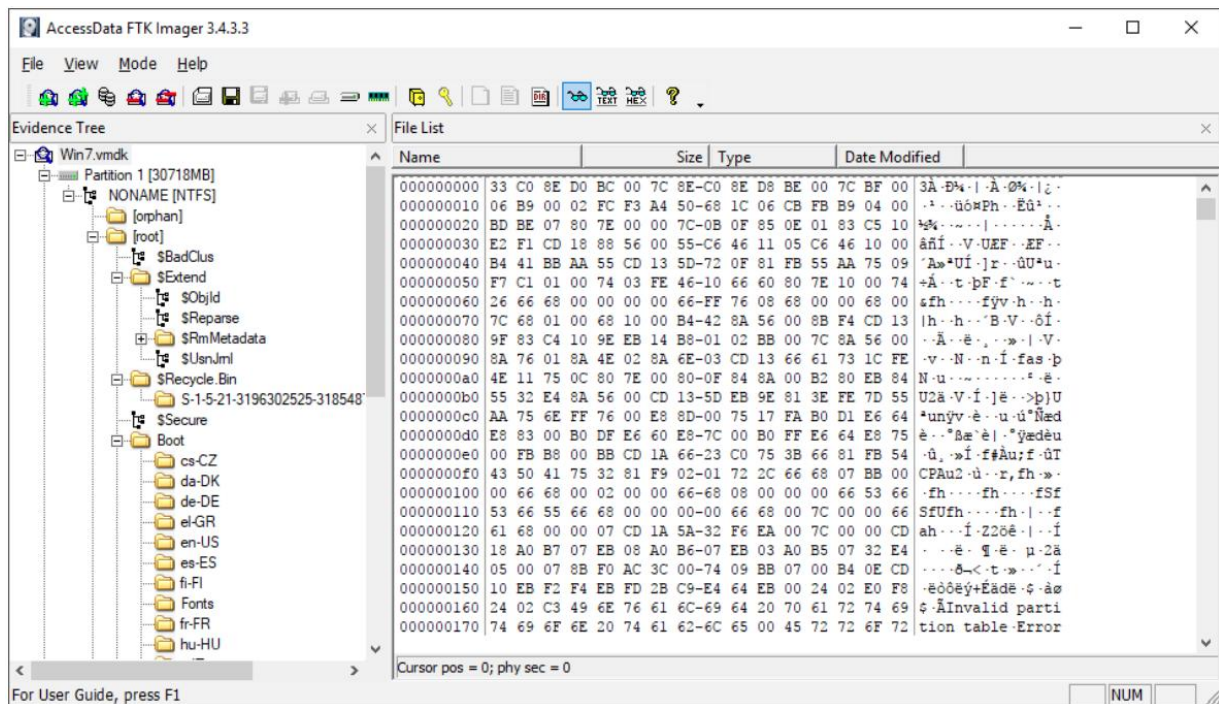
1. Thực hiện phân tích dựa trên dữ liệu ổ đĩa (tự chọn) sử dụng Autopsy
 - Chọn tìm các số điện thoại và địa chỉ IP có trong Filesystem.
 - Thực hiện việc xem xét toàn bộ Filesystem, xem xét các lựa chọn nằm ở phía bên trái của màn hình.
 - Tìm thư mục có nhiều File nhất trong Filesystem.
 - Xem các file hình ảnh chứa trong Filesystem bằng chế độ View Thumbnail. Xác định số lượng các files dạng doc và pdf chứa trong Filesystem.
 - Sử dụng nút "Generate Report" để tạo ra báo cáo dạng HTML và Excel, xem nội dung báo cáo trong mục Report. Nêu nhận xét, kết luận về nội dung của báo cáo.

b) Phân tích dữ liệu bằng công cụ FTK Imager

Bước 1: Chọn File => Add Evidence Item để chọn chứng cứ cần thêm.



Bước 2: Mục tiêu trong ví dụ này là file dữ liệu ổ đĩa của máy ảo Windows 7. Sinh viên chọn một file đĩa phù hợp để thực hành sử dụng công cụ.



Có thể dễ dàng xem được cây hệ thống của file dữ liệu của ổ đĩa được chọn.

B3. File System Forensics

B2.1. Tổng quan

B2.1.1. Định nghĩa

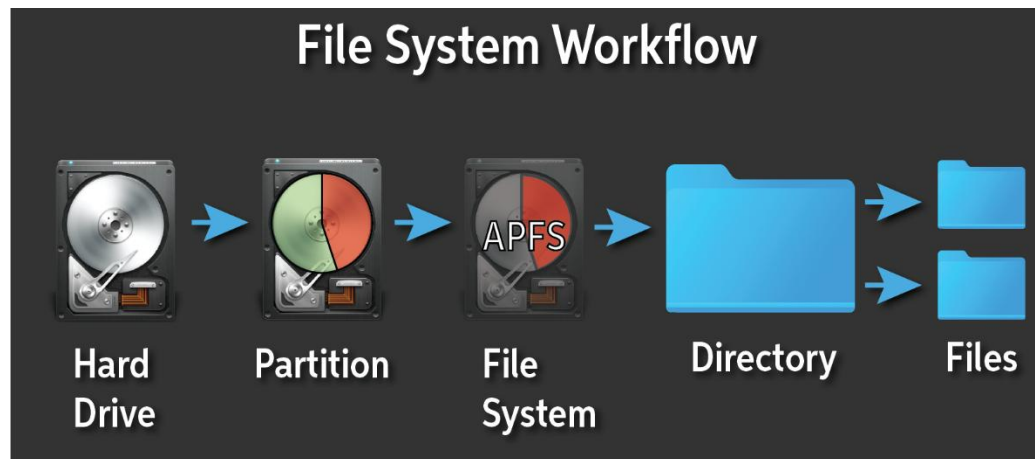
File System (hệ thống tập tin) là phần mềm hoặc hệ thống quản lý, tổ chức và lưu trữ dữ liệu trên các thiết bị lưu trữ như đĩa cứng, ổ USB hoặc thẻ nhớ. Nó cung cấp giao diện cho người dùng truy cập và quản lý tập tin, thư mục, đồng thời đảm bảo tính toàn vẹn và bảo mật dữ liệu thông qua kiểm soát quyền truy cập và các tính năng sao lưu, khôi phục.

B2.1.2. Tầm quan trọng của File System

File System đóng vai trò quan trọng trong việc:

- **Tổ chức và lưu trữ dữ liệu hiệu quả:** Giúp người dùng dễ dàng truy cập và sử dụng dữ liệu.
- **Bảo vệ tính toàn vẹn của dữ liệu:** Giới hạn quyền truy cập và cung cấp các tính năng sao lưu, khôi phục để tránh mất mát hoặc hư hỏng dữ liệu.
- **Tối ưu hóa hiệu suất hệ thống:** Tổ chức tập tin theo cách đặc biệt để giảm thời gian tìm kiếm và truy cập, cải thiện hiệu suất máy tính.

Việc lựa chọn đúng loại File System phù hợp với nhu cầu sử dụng là yếu tố then chốt để đảm bảo sự ổn định và tin cậy của hệ thống.



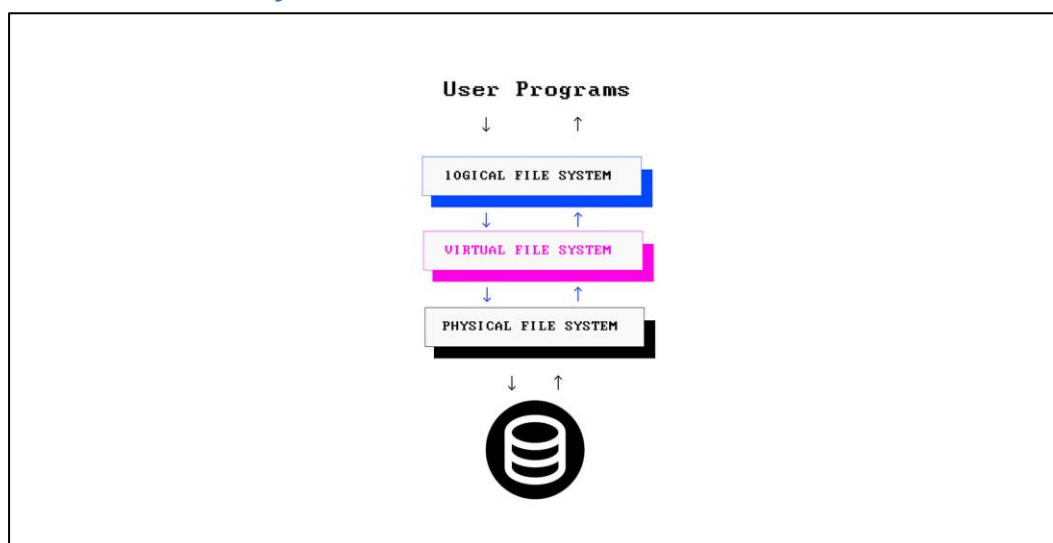
(Nguồn: Sweetwater)

B2.1.3. Các thành phần của File System

File System bao gồm các thành phần cơ bản sau:

- **Boot Sector**: Phần đầu tiên được đọc khi khởi động, chứa thông tin cần thiết về kích thước phân vùng và loại hệ thống tập tin.
- **File Allocation Table (FAT) hoặc Master File Table (MFT)**: Bảng chứa thông tin về các tệp tin và thư mục. FAT được dùng trong hệ thống tập tin đơn giản như FAT32, còn MFT trong hệ thống nâng cao như NTFS.
- **Directory**: Cấu trúc chứa các thư mục và tệp tin, giúp tổ chức và quản lý dữ liệu có hệ thống.
- **File System Drivers**: Chương trình phần mềm kết nối hệ thống tập tin với phần cứng lưu trữ, cho phép truy cập và ghi dữ liệu.
- **System Call Interface**: Giao diện sử dụng các lệnh hệ thống trong ứng dụng để truy cập tính năng của hệ thống tập tin.

B2.1.4. Cấu trúc File System



(Nguồn: freeCodeCamp)

Cấu trúc của File System bao gồm:

- **Partition (Phân vùng):** Phần của ổ đĩa được chia ra để tạo các vùng lưu trữ độc lập, mỗi phân vùng có thể được định dạng với một loại hệ thống tập tin khác nhau, giúp tối ưu hóa hiệu suất lưu trữ.
- **Block:** Đơn vị nhỏ nhất dùng để lưu dữ liệu. Khi một tệp được lưu trữ, hệ thống sẽ chia nội dung của nó thành nhiều block. Một số block đặc biệt trong hệ thống tập tin bao gồm:
 - **Boot Block:** chứa thông tin khởi động.
 - **Super Block:** lưu các thông tin tổng quát về hệ thống tập tin.
 - **Data Block:** nơi chứa dữ liệu thực tế của tệp.
 - **Free Block List:** quản lý các block trống có thể sử dụng.
- **Inode:** Một cấu trúc dữ liệu lưu trữ thông tin về tệp hoặc thư mục. Nó không chứa nội dung tệp mà chỉ lưu các thuộc tính như kích thước, quyền truy cập, thời gian tạo, sửa đổi và danh sách block lưu trữ nội dung tệp. Mỗi tệp hay thư mục đều có một inode riêng biệt để quản lý.
- **File:** Đơn vị lưu trữ dữ liệu do người dùng tạo ra. Tệp có thể thuộc nhiều loại khác nhau như văn bản, hình ảnh, âm thanh hay tệp thực thi. Mỗi tệp có một inode để quản lý thông tin và sử dụng một hoặc nhiều block để lưu nội dung.

B2.1.5. Các loại File System

Có nhiều loại File System khác nhau, mỗi loại có đặc điểm và tính năng riêng. Sau đây là một số loại File System phổ biến:

- **EXT (Extended File System):** Hệ thống tập tin được sử dụng trên các hệ điều hành dựa trên Linux, bao gồm các phiên bản như EXT2, EXT3 và EXT4. Hệ thống này cung cấp khả năng bảo mật, quản lý tập tin hiệu quả và hỗ trợ journaling để đảm bảo tính toàn vẹn dữ liệu.
- **FAT (File Allocation Table):** Hệ thống tập tin đơn giản, hỗ trợ trên nhiều hệ điều hành như Windows, Linux và macOS. FAT12, FAT16 và FAT32 thường được sử dụng trên các thiết bị lưu trữ nhỏ như thẻ nhớ, ổ USB. FAT32 hỗ trợ tối đa 2TB nhưng không hỗ trợ các tính năng bảo mật nâng cao.
- **NTFS (New Technology File System):** Hệ thống tập tin phổ biến trên Windows, cung cấp nhiều tính năng mạnh mẽ như phân quyền bảo mật, nén dữ liệu, mã hóa, hỗ trợ tệp có kích thước lớn và tính năng journaling giúp phục hồi dữ liệu khi hệ thống gặp sự cố.
- **HFS+ (Hierarchical File System Plus):** Hệ thống tập tin trước đây được sử dụng trên macOS và iOS, hỗ trợ quản lý dữ liệu hiệu quả nhưng đã bị thay thế bởi APFS trên các thiết bị Apple hiện đại.

- **APFS (Apple File System):** Hệ thống tập tin được phát triển bởi Apple, tối ưu cho các thiết bị sử dụng SSD. APFS cải thiện hiệu suất, bảo mật, hỗ trợ snapshot và mã hóa dữ liệu mạnh mẽ.
- **UFS (Unix File System):** Hệ thống tập tin tiêu chuẩn trên Unix và FreeBSD, cung cấp hiệu suất ổn định và độ tin cậy cao.

Bài tập (yêu cầu làm)

2. Ngoài các File System phổ biến đã liệt kê trên, còn những hệ thống tập tin nào khác được sử dụng trên các nền tảng khác nhau? Hãy liệt kê và mô tả ngắn gọn đặc điểm của chúng.

B2.2. Windows File System

Windows hỗ trợ nhiều hệ thống tập tin khác nhau, mỗi loại có đặc điểm riêng về hiệu suất, bảo mật và khả năng quản lý dữ liệu. Một số hệ thống tập tin phổ biến bao gồm: **FAT32**, **exFAT**, **NTFS**, **ReFS (Resilient File System)** - được Microsoft thiết kế để tối ưu hóa cho lưu trữ dữ liệu lớn và khả năng chống lỗi tự động.

Trong phần này, sẽ tập trung vào ba hệ thống tập tin phổ biến nhất của Windows: **FAT**, **exFAT** và **NTFS**, phân tích kiến trúc và cơ chế quản lý dữ liệu của chúng.

B2.2.1. Kiến trúc FAT

FAT (File Allocation File) là một hệ thống tập tin đơn giản, ban đầu được thiết kế cho các ổ đĩa nhỏ và cấu trúc thư mục đơn giản.

Hệ thống này được đặt tên theo phương pháp tổ chức của nó – **Bảng phân bố tập tin (File Allocation Table - FAT)**, nằm ở phần đầu của phân vùng. Để đảm bảo tính toàn vẹn của dữ liệu, hệ thống FAT lưu trữ hai bản sao của bảng này để có thể khôi phục trong trường hợp một bản bị hỏng.

Ngoài ra, bảng FAT và thư mục gốc (root folder) phải được lưu trữ tại một vị trí cố định trên đĩa để hệ thống có thể định vị chính xác các tệp tin quan trọng cần thiết cho quá trình khởi động.

Một phân vùng được định dạng với hệ thống FAT được chia thành các **cụm (clusters)**. Kích thước cụm mặc định được xác định dựa trên dung lượng phân vùng. Trong hệ thống FAT, số cụm phải vừa với 16 bit và là lũy thừa của hai.

* Cấu trúc của một phân vùng FAT

Partition Boot Sector	FAT1	FAT2 (duplicate)	Root folder	Other folders and all files.
-----------------------	------	------------------	-------------	------------------------------

(Nguồn: ntfs.com)

Hệ thống FAT tổ chức phân vùng theo cấu trúc bao gồm các thành phần chính như sau:

- **Boot Sector:** Chứa thông tin quan trọng về hệ thống tập tin, bao gồm kích thước phân vùng, kích thước cụm, vị trí của bảng FAT và mã khởi động nếu phân vùng có thể boot được.
- **FAT1:** Bảng quản lý vị trí của các khối dữ liệu trên đĩa. Theo dõi các cụm trống, các cụm bị lỗi và chuỗi cụm thuộc về một tệp tin hoặc thư mục.
- **FAT2:** Là bản sao của FAT1, được sử dụng để khôi phục dữ liệu trong trường hợp bảng FAT1 bị hỏng.
- **Root folder:** Chứa danh sách các tệp tin và thư mục con. Trên FAT12 và FAT16, thư mục gốc có kích thước cố định và nằm ở một vị trí cố định trên đĩa. Trên FAT32, thư mục gốc có thể thay đổi vị trí và không bị giới hạn kích thước.
- **Other Folders and All Files:** Khu vực chứa nội dung thực tế của các tệp tin và thư mục con. Được tổ chức thành các **cụm (clusters)**, với kích thước cụm thay đổi tùy thuộc vào kích thước phân vùng.

Cấu trúc này đảm bảo hệ thống có thể quản lý tập tin một cách đơn giản nhưng vẫn hỗ trợ các yêu cầu cơ bản về lưu trữ và truy xuất dữ liệu.

* Sự khác biệt giữa các hệ thống FAT

Hệ thống tập tin FAT có ba phiên bản chính: FAT12, FAT16, và FAT32. Mỗi phiên bản có sự khác biệt về số byte trên mỗi mục nhập trong bảng cấp phát tệp (FAT) và giới hạn số lượng cụm. Bảng sau mô tả chi tiết:

FAT Type	Số byte trên mỗi mục nhập trong FAT	Giới hạn số cụm
FAT12	1.5 byte	Dưới 4,087 cụm
FAT16	2 byte	Từ 4,087 đến 65,526 cụm
FAT32	4 byte	Từ 65,526 đến 268,435,456 cụm

Hệ thống **FAT12** được sử dụng cho các thiết bị lưu trữ có dung lượng nhỏ, chẳng hạn như đĩa mềm (floppy disk). **FAT16** phổ biến trên các ổ đĩa cứng thời kỳ đầu, còn **FAT32** hỗ trợ dung lượng lớn hơn và hiệu suất cao hơn, phù hợp cho các thiết bị lưu trữ hiện đại như USB, thẻ nhớ và ổ cứng ngoài.

Bài tập (yêu cầu làm)

3. Thực hiện phân tích FAT32 filesystem và hoàn thành các câu hỏi tại <https://tryhackme.com/room/fat32analysis>

Gợi ý: <https://www.youtube.com/watch?v=0jMoPhiNeVk>

B2.2.2. Kiến trúc exFAT

exFAT (Extended File Allocation Table) là hệ thống tệp do Microsoft phát triển để khắc phục hạn chế của FAT32, đặc biệt hỗ trợ tệp có kích thước lớn hơn 4GB. Nó

tối ưu hóa cho thiết bị lưu trữ flash như USB, thẻ SD nhưng vẫn đảm bảo hiệu suất cao hơn trên ổ đĩa cứng di động.

Một phân vùng exFAT bao gồm các thành phần chính sau:

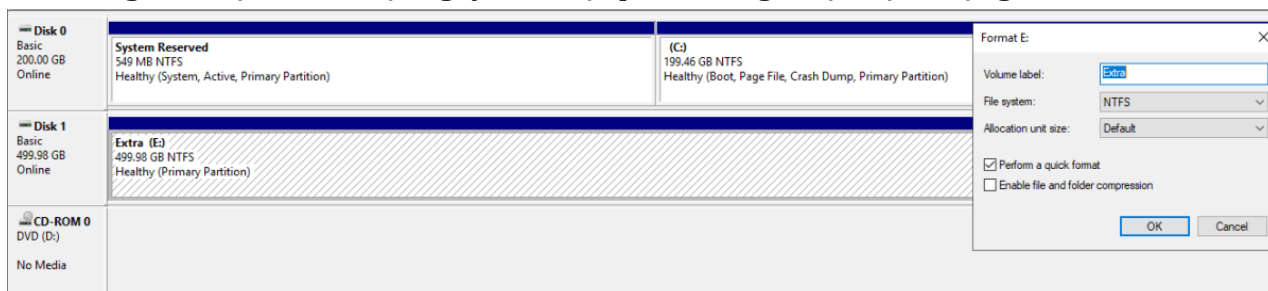
- **Boot Sector:** Chứa thông tin về hệ thống tệp, thông số phân vùng và vị trí của các bảng quan trọng. Không có bảng phân vùng riêng mà chỉ sử dụng **BIOS Parameter Block (BPB) mở rộng**.
- **FAT:** Chỉ có **một bảng FAT duy nhất** (khác với FAT16/FAT32 có hai bản sao). Hỗ trợ **Cluster Bitmap** để theo dõi các cụm dữ liệu đã sử dụng, giảm thời gian tìm kiếm vùng trống. Sử dụng **entry 32-bit**, hỗ trợ kích thước phân vùng lên đến **128 PiB**.
- **Allocation Bitmap:** Một bitmap giúp theo dõi tình trạng sử dụng của các cụm dữ liệu, giảm phân mảnh và tối ưu tốc độ ghi trên thiết bị flash.
- **Uppcase Table:** Bảng quy đổi chữ hoa, hỗ trợ chuẩn hóa tên tệp để hệ thống không phân biệt chữ hoa và chữ thường khi so sánh tên tệp.
- **Root Directory:** Không nằm ở vị trí cố định như FAT16/FAT32 mà có thể di chuyển trong phân vùng, giúp tối ưu dung lượng lưu trữ. Tệp thư mục có thể mở rộng mà không bị giới hạn kích thước như trong FAT32.
- **Data Region:** Khu vực chính để lưu trữ tệp và thư mục. Hỗ trợ kích thước cụm (**cluster size**) từ **4KB đến 32MB**, giúp giảm phân mảnh dữ liệu.

B2.2.3. Kiến trúc NTFS

NTFS (New Technology File System) là một hệ thống tệp tin độc quyền do Microsoft phát triển. Được giới thiệu lần đầu tiên vào năm 1993 trong Windows NT 3.1, NTFS đã trải qua nhiều cải tiến nhưng vẫn giữ nguyên các nguyên tắc cốt lõi.

NTFS Là hệ thống tệp tin tiêu chuẩn của các hệ điều hành Windows NT (bao gồm Windows 2000, XP, Vista, 7, 8, 10 và các phiên bản sau này), NTFS cung cấp sự kết hợp giữa độ tin cậy, hiệu suất và khả năng mở rộng. Chính vì những ưu điểm này, Microsoft chưa từng cảm thấy cần thiết phải thay thế NTFS ngay cả sau hơn 27 năm kể từ khi ra mắt.

Trong kiến trúc hệ thống, NTFS hoạt động dưới dạng một trình điều khiển nhân (Kernel Mode Driver) có tên **NTFS.sys**. Trình điều khiển này chạy ngầm trong hệ thống và được kích hoạt ngay khi một phân vùng được định dạng theo NTFS.



*** Yêu cầu kiến thức trước khi thực hiện:**

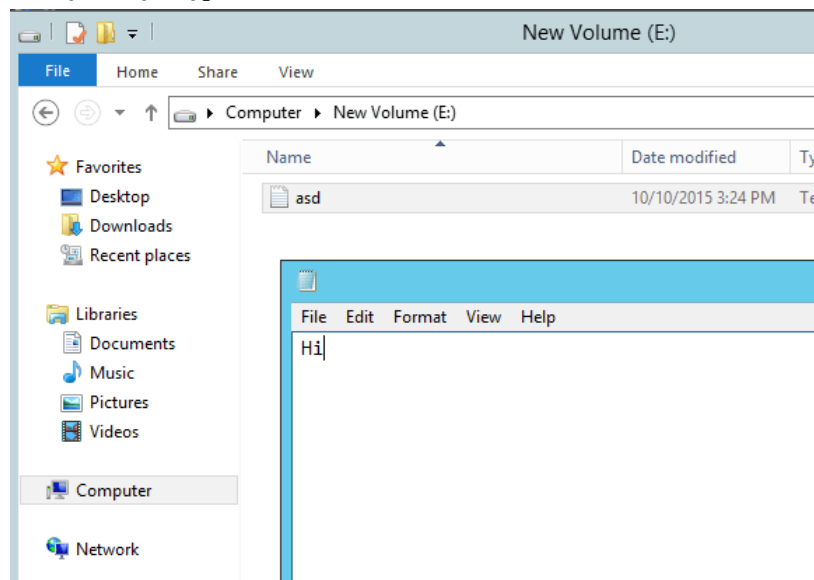
Để thực hiện nội dung này, cần có kiến thức cơ bản về hệ thống lưu trữ của Windows. Khuyến nghị nên tìm hiểu các khái niệm sau trước khi tiếp tục:

- Tìm hiểu về **Master Boot Record (MBR)**
- Tìm hiểu về **Master Boot Sector (MBS)**
- **Cấu trúc lưu trữ của Windows (Windows Storage Stack)**

*** Cách tạo tập tin trong phân vùng NTFS (nguồn: knowitlikepro):**

Khi NTFS lưu trữ một tệp, nó bắt đầu bằng cách tạo một **đoạn bản ghi tệp (file record segment)** có kích thước **1KB**, được gọi là **bản ghi cơ sở (base record)**.

Tất cả các tệp, kể cả các tệp hệ thống ẩn như **\$MFT**, **\$LOGFILE**, **\$VOLUME**, đều bắt đầu bằng một bản ghi cơ sở. Khi nhắc đến **MFT (Master File Table)**, thực chất là đang nói đến toàn bộ danh sách các bản ghi cơ sở và các bản ghi con (child record) của tất cả các tệp trong phân vùng NTFS.

Bước 1: Tạo một phân vùng NTFS mới**Bước 2:** Tạo một tập tin văn bản có kích thước < 1KB.**Bước 3:** Kiểm Tra MBR và MBS Bằng Hex Editor.

Có nhiều công cụ miễn phí hỗ trợ, một trong những công cụ phổ biến là **HxD**, có thể tải về từ: <https://mh-nexus.de/en/hxd/>

- MBR:

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text	
00000000	33	C0	8E	D0	BC	00	7C	8E	C0	8E	D8	BE	00	7C	BF	00	3ĂZĐ4. ŽĂZĐ4. ž.	Sector 0
00000010	06	B9	00	02	FC	F3	A4	50	68	1C	06	CB	FB	B9	04	00	. ' . . úó×Ph. .Ěú' . .	
00000020	BD	BE	07	80	7E	00	00	7C	0B	0F	85	0E	01	83	C5	10	ž4. Ě~ fĂ.	
00000030	E2	F1	CD	18	88	56	00	55	C6	46	11	05	C6	46	10	00	ăńí. ^V.UEF. .ĚF. .	
00000040	B4	41	BB	AA	55	CD	13	5D	72	0F	81	FB	55	AA	75	09	'A»*Uí. r. .úU*u.	
00000050	F7	C1	01	00	74	03	FE	46	10	66	60	80	7E	10	00	74	-Ă. .t. pF. f' Ě~ . . t	
00000060	26	66	68	00	00	00	00	66	FF	76	08	68	00	00	68	00	&fh. . . . fŷv. h. . h.	
00000070	7C	68	01	00	68	10	00	B4	42	8A	56	00	8B	F4	CD	13	h. . h. . 'BŠV. <óí.	
00000080	9F	83	C4	10	9E	EB	14	B8	01	02	BB	00	7C	8A	56	00	ŷfĂ. žž. . . . » . ŠV.	
00000090	8A	76	01	8A	4E	02	8A	6E	03	CD	13	66	61	73	1C	FE	Šv. ŠN. Šn. í. fas. p	
000000A0	4E	11	75	0C	80	7E	00	80	0F	84	8A	00	B2	80	EB	84	N. u. Ě~ . Ě. . . Š. ' ĚĚ. .	
000000B0	55	32	E4	8A	56	00	CD	13	5D	EB	9E	81	3E	FE	7D	55	U2ăŠV. í. žž. >p) U	
000000C0	AA	75	6E	FF	76	00	E8	8D	00	75	17	FA	B0	D1	E6	64	'unŷv. Ě. . . u. ú' Năd	
000000D0	E8	83	00	B0	DF	E6	60	E8	7C	00	B0	FF	E6	64	E8	75	Ěf. 'ăă' ĚĚ. . ' ŷădĚu	
000000E0	00	FB	B8	00	BB	CD	1A	66	23	C0	75	3B	66	81	FB	54	. ŷ. . » í. f#Ău; f. ŷŮT	
000000F0	43	50	41	75	32	81	F9	02	01	72	2C	66	68	07	BB	00	CPAu2. ŷ. . . r. fh. ».	
00000100	00	66	68	00	02	00	00	66	68	08	00	00	00	66	53	66	. fh. . . . fh. . . . fSf	
00000110	53	66	55	66	68	00	00	00	00	66	68	00	7C	00	00	66	SfUfh. . . . fh. . . f	
00000120	61	68	00	00	07	CD	1A	5A	32	F6	EA	00	7C	00	00	CD	ah. . . . í. Z2žĚ. . . í	
00000130	18	A0	B7	07	EB	08	A0	B6	07	EB	03	A0	B5	07	32	E4	. . . Ě. 1. Ě. p. 2ă	
00000140	05	00	07	8B	F0	AC	3C	00	74	09	BB	07	00	B4	0E	CD	. . . <ă-<. t. » . . ' í	
00000150	10	EB	F2	F4	EB	FD	2B	C9	E4	64	EB	00	24	02	E0	F8	. ĚžžĚŷ+ĚăĚĚ. \$. àž	
00000160	24	02	C3	49	6E	76	61	6C	69	64	20	70	61	72	74	69	\$. ĂInvalid parti	
00000170	74	69	6F	6E	20	74	61	62	6C	65	00	45	72	72	6F	72	tion table. Error	
00000180	20	6C	6F	61	64	69	6E	67	20	6F	70	65	72	61	74	69	loading operati	
00000190	6E	67	20	73	79	73	74	65	6D	00	4D	69	73	73	69	6E	ng system. Missin	
000001A0	67	20	6F	70	65	72	61	74	69	6E	67	20	73	79	73	74	g operating syst	
000001B0	65	6D	00	00	00	63	7B	9A	7A	6A	79	89	00	00	00	02	em. . . . c(šžzyk. . . .	
000001C0	03	00	07	FE	3F	81	80	00	00	00	00	E8	1F	00	00	00	. . . p? . Ě. . . Ě. . . .	
000001D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
000001E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
000001F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	55	AA	 U*	

Dựa vào hình minh họa trên, **sector đầu tiên của phân vùng có địa chỉ 0x80 (Hex). 0x80 trong hệ thập phân = 128**, nghĩa là MBS nằm ở sector 128. Nhảy đến **sector 128** để tìm MBS trong Hex Editor.

80 00 00 = 00 00 80

00 00 80 = 0x80 (Hex)

0x80 (Hex) = 128 (Decimal)

00010040	F6	00	00	00	01	00	00	00	73	7E	1C	BC	BD	1C	BC	F2	ž. s~. ž4. žž
00010050	00	00	00	00	FA	33	C0	8E	D0	BC	00	7C	FB	68	C0	07	 úĂZĐ4. žhĂ.
00010060	1F	1E	68	66	00	CB	88	16	0E	00	66	81	3E	03	00	4E	. . hf. Ě^ . . . f. > . . N
00010070	54	46	53	75	15	B4	41	BB	AA	55	CD	13	72	0C	81	FB	TFSu. 'A»*Uí. r. . ŷ
00010080	55	AA	75	06	F7	C1	01	00	75	03	E9	DD	00	1E	83	EC	U*u. +Ă. u. Ěŷ. . fi
00010090	18	68	1A	00	B4	48	8A	16	0E	00	8B	F4	16	1F	CD	13	. h. . 'HŠ. . . . <ă. í.
000100A0	9F	83	C4	18	9E	58	1F	72	E1	3B	06	0B	00	75	DB	A3	ŷfĂ. žX. rá; . . . uŷĚ
000100B0	0F	00	C1	2E	0F	00	04	1E	5A	33	DB	B9	00	20	2B	C8	. . Ă. Z3Ů. . +Ě
000100C0	66	FF	06	11	00	03	16	0F	00	8E	C2	FF	06	16	00	E8	fŷ. ŽĂŷ. . . Ě
000100D0	4B	00	2B	C8	77	EF	B8	00	BB	CD	1A	66	23	C0	75	2D	K. +Ěwi. . » í. f#Ău-
000100E0	66	81	FB	54	43	50	41	75	24	81	F9	02	01	72	1E	16	f. ŷŮTCPAu\$. ŷ. . . r. .
000100F0	68	07	BB	16	68	52	11	16	68	09	00	66	53	66	53	66	h. » . hR. . h. . fSfSf
00010100	55	16	16	16	68	B8	01	66	61	0E	07	CD	1A	33	C0	BF	U. . . h. . fa. . í. 3Ăž
00010110	0A	13	B9	F6	0C	FC	F3	AA	E9	FE	01	90	90	66	60	1E	. . 'ž. ŷžž. Ěp. . . f'.
00010120	06	66	A1	11	00	66	03	06	1C	00	1E	66	68	00	00	00	. f; . . f. . . . fh. . .
00010130	00	66	50	06	53	68	01	00	68	10	00	B4	42	8A	16	0E	. fp. Šh. h. . 'BŠ. .
00010140	00	16	1F	8B	F4	CD	13	66	59	5B	5A	66	59	66	59	1F	. . . <óí. fŷ[Zfŷfŷ.
00010150	0F	82	16	00	66	FF	06	11	00	03	16	0F	00	8E	C2	FF	 fŷ. ŽĂŷ
00010160	0E	16	00	75	BC	07	1F	66	61	C3	A1	F6	01	E8	09	00	. . . u4. . faĂ; ž. Ě. .
00010170	A1	FA	01	E8	03	00	F4	EB	FD	8B	F0	AC	3C	00	74	09	ž. ŷ. Ě. žĚŷ<ă-<. t.
00010180	B4	0E	BB	07	00	CD	10	EB	F2	C3	0D	0A	41	20	64	69	' í. ĚžĂ. . A di
00010190	73	6B	20	72	65	61	64	20	65	72	72	6F	72	20	6F	63	sk read error oc
000101A0	63	75	72	72	65	64	00	0D	0A	42	4F	4F	54	4D	47	52	curréd. . . BOOTMGR
000101B0	20	69	73	20	63	6F	6D	70	72	65	73	73	65	64	00	0D	is compressed. .
000101C0	0A	50	72	65	73	73	20	43	74	72	6C	2B	41	6C	74	2B	. Press Ctrl+Alt+
000101D0	44	65	6C	20	74	6F	20	72	65	73	74	61	72	74	0D	0A	Del to restart. .
000101E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000101F0	00	00	00	00	00	00	8A	01	A7	01	BF	01	00	00	55	AA	 Š. \$. Ě. . . U*

- MFT:

Xác định vị trí của MFT theo thông tin được tô đậm dưới đây:

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00010000	EB	52	90	4E	54	46	53	20	20	20	20	00	02	08	00	00	ẽR.NTFS
00010010	00	00	00	00	00	F8	00	00	3F	00	FF	00	80	00	00	00	ø...?..ÿ.€...
00010020	00	00	00	00	80	00	80	00	FF	E7	1F	00	00	00	00	00	Ë.ÿç.....
00010030	55	54	01	00	00	00	00	00	02	00	00	00	00	00	00	00	UT.....
00010040	F6	00	00	00	01	00	00	00	73	7E	1C	BC	BD	1C	BC	F2	ö.....s~.l&g.kò
00010050	00	00	00	00	FA	33	C0	8E	D0	BC	00	7C	FB	68	C0	07	ú3ÀŽĐ+.jùhÀ.
00010060	1F	1E	68	66	00	CB	88	16	0E	00	66	81	3E	03	00	4E	..hf.È^...f.>..N
00010070	54	46	53	75	15	B4	41	BB	AA	55	CD	13	72	0C	81	FB	TFSu.'A»*Uí.r..û
00010080	55	AA	75	06	F7	C1	01	00	75	03	E9	DD	00	1E	83	EC	U*u.÷Á..u.éY..fi
00010090	18	68	1A	00	B4	48	8A	16	0E	00	8B	F4	16	1F	CD	13	.h..'HŠ...<ô..í.
000100A0	9F	83	C4	18	9E	58	1F	72	E1	3B	06	0B	00	75	DB	A3	ÝfÄ.žX.rá;...uŮz
000100B0	0F	00	C1	2E	0F	00	04	1E	5A	33	DB	B9	00	20	2B	C8	..Ä.....Z3Ů¹. +È
000100C0	66	FF	06	11	00	03	16	0F	00	8E	C2	FF	06	16	00	E8	fý.....ŽÄÿ...è
000100D0	4B	00	2B	C8	77	EF	B8	00	BB	CD	1A	66	23	C0	75	2D	K.+Èwì,..»í.f#Àu-
000100E0	66	81	FB	54	43	50	41	75	24	81	F9	02	01	72	1E	16	f.ùTCPAu\$.ù...r.
000100F0	68	07	BB	16	68	52	11	16	68	09	00	66	53	66	53	66	h.».hR..h..fSfSf
00010100	55	16	16	16	68	B8	01	66	61	0E	07	CD	1A	33	C0	BF	U...h..fa..í.3Ä¿
00010110	0A	13	B9	F6	0C	FC	F3	AA	E9	FE	01	90	90	66	60	1E	..²ò.úó*ép...f`.
00010120	06	66	A1	11	00	66	03	06	1C	00	1E	66	68	00	00	00	.f;...f.....fh...
00010130	00	66	50	06	53	68	01	00	68	10	00	B4	42	8A	16	0E	.fP.Sh..h..'BŠ..
00010140	00	16	1F	8B	F4	CD	13	66	59	5B	5A	66	59	66	59	1F	...<óí.fY[ZfYfY.
00010150	0F	82	16	00	66	FF	06	11	00	03	16	0F	00	8E	C2	FF	...fý.....ŽÄÿ
00010160	0E	16	00	75	BC	07	1F	66	61	C3	A1	F6	01	E8	09	00	...u*.faÄ;ö.è..
00010170	A1	FA	01	E8	03	00	F4	EB	FD	8B	F0	AC	3C	00	74	09	jú.è..ôëý<ô-<t.
00010180	B4	0E	BB	07	00	CD	10	EB	F2	C3	0D	0A	41	20	64	69	`.»..í.èöÄ..A di
00010190	73	6B	20	72	65	61	64	20	65	72	72	6F	72	20	6F	63	sk read error oc
000101A0	63	75	72	72	65	64	00	0D	0A	42	4F	4F	54	4D	47	52	curred...BOOTMGR
000101B0	20	69	73	20	63	6F	6D	70	72	65	73	73	65	64	00	0D	is compressed..
000101C0	0A	50	72	65	73	73	20	43	74	72	6C	2B	41	6C	74	2B	.Press Ctrl+Alt+
000101D0	44	65	6C	20	74	6F	20	72	65	73	74	61	72	74	0D	0A	Del to restart..
000101E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000101F0	00	00	00	00	00	00	8A	01	A7	01	BF	01	00	00	55	AA	Š.S.¿...U*

• Chuyển Đổi Giá Trị Little Endian

Giá trị hex ban đầu được lưu trữ ở dạng **Little Endian**:

55 54 01 00 00 00 00 00

Đảo ngược lại thứ tự byte để chuyển về dạng **Big Endian**:

00 00 00 00 00 01 54 55 → 0x015455 (Hex)

Chuyển đổi giá trị này sang hệ thập phân:

0x015455 = 87125 (Decimal)

Tức là **MFT bắt đầu từ cluster 87125**.

* Có thể sử dụng *Data inspector* trong công cụ HxD để lấy thông tin này

Data inspector	
◀◀▶▶	
Binary (8 bit)	01010101
Int8	go to: 85
UInt8	go to: 85
Int16	go to: 21589
UInt16	go to: 21589
Int24	go to: 87125
UInt24	go to: 87125
Int32	go to: 87125
UInt32	go to: 87125
Int64	go to: 87125
UInt64	go to: 87125
AnsiChar / char8_t	U
WideChar / char16_t	嘔
UTF-8 code point	U (U+0055)
Single (float32)	1.220881287043E-40
Double (float64)	4.30454693939186E-319
OLETIME	12/30/1899
FILETIME	1/1/1601 12:00:00 AM
DOS date	2/21/2022
DOS time	10:34:42 AM
DOS time & date	Invalid
time_t (32 bit)	1/2/1970 12:12:05 AM
time_t (64 bit)	1/2/1970 12:12:05 AM
GUID	Invalid
Disassembly (x86-16)	push bp
Disassembly (x86-32)	push ebp
Disassembly (x86-64)	push rbp

- **Xác định sector chứa MFT**

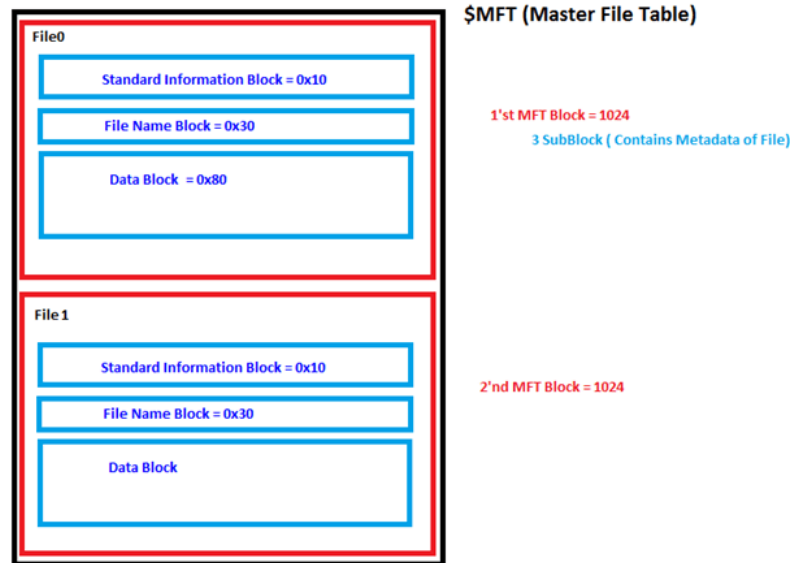
Trong hệ thống tệp NTFS, **mỗi cluster chứa 8 sector**, vì vậy cần nhân giá trị cluster với 8 để tính ra sector: $87125 \times 8 = 697000$

Giá trị **697000** là **LBA (Logical Block Address)** trong phân vùng. Tuy nhiên, để tính toán **vị trí thực tế trên đĩa**, cần **cộng thêm số sector trước đó trong phân vùng**.

Giả sử **Master Boot Sector (MBS)** bắt đầu từ **sector 128**, vị trí chính xác của MFT trên ổ đĩa sẽ là: $697000 + 128 = 697128$

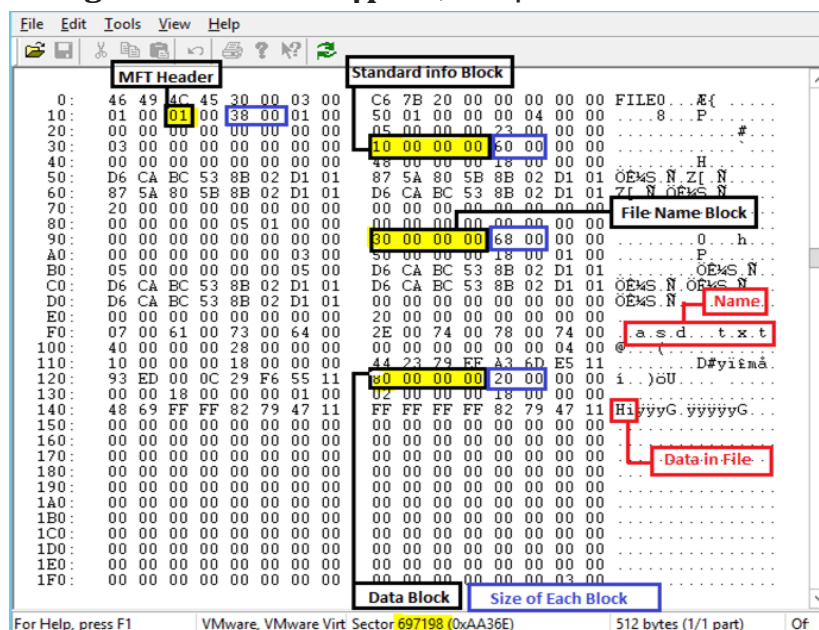
Vậy **MFT bắt đầu từ sector 697128 trên đĩa**.

a) Phân tích MFT



- Mỗi block trong MFT bắt đầu bằng tên đặc trưng: **FILE0**
- 35 block đầu tiên** của MFT được NTFS sử dụng cho hệ thống, tức là từ block 0 đến 34.
- Block thứ 35** của MFT lưu trữ thông tin về tập tin dữ liệu (Data File).
- Mỗi block trong MFT có kích thước cố định là **1024 byte** (1KB).
- Nếu vị trí hiện tại là **697128**, thì **block đầu tiên dành cho dữ liệu người dùng sẽ bắt đầu tại vị trí: 697128+70=697198**
 - Lý do: Mỗi bản ghi MFT được lưu trong 2 sector, mỗi sector có kích thước 512 byte, do đó: $35 \times 2 = 70$ sectors.

Sau khi khởi tạo volume, NTFS sẽ tự động tạo thêm các thư mục hệ thống mặc định như "**System Volume Information**". Do đó, để tìm tập tin cần kiểm tra, cần quét **Entry trong MFT chứa tên tập tin**, ví dụ: *asd.txt*.



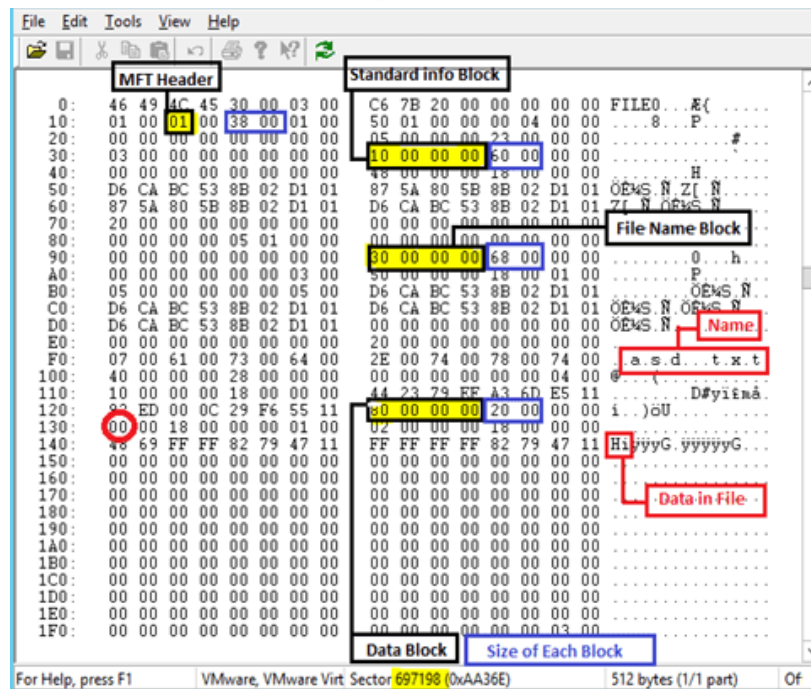
* Phân loại tập tin trong MFT:

• Tập tin Resident:

Khi các thuộc tính của tập tin có thể vừa trong bản ghi MFT, tập tin đó được gọi là Resident File.

- Thông tin về tập tin như filename, timestamp luôn là resident attributes.
- Dữ liệu của tập tin cũng được lưu hoàn toàn trong MFT (1KB record), không cần tham chiếu đến vị trí khác trên ổ đĩa.
- Ký hiệu Resident File trong Hex Editor:

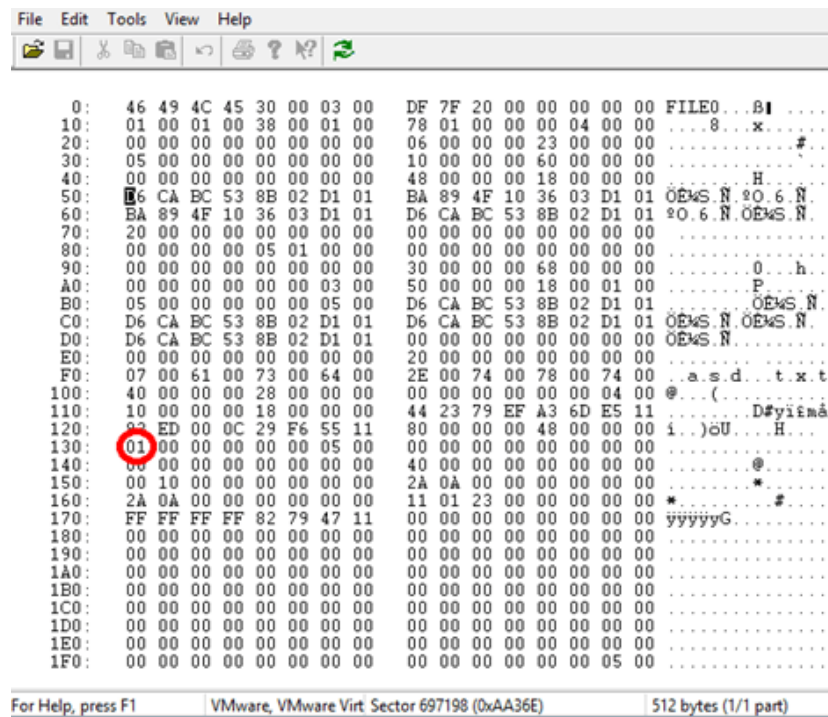
Giá trị **00** thể hiện tập tin là Resident.



• Tập tin Non-Resident

Khi dữ liệu tập tin quá lớn để lưu trữ trong một bản ghi MFT, NTFS sẽ **chuyển dữ liệu sang khu vực khác trên ổ đĩa**.

- Các thuộc tính dữ liệu Non-Resident được lưu ở các cluster riêng biệt.
- NTFS sử dụng “Attribute List” để lưu vị trí của các phần dữ liệu này.
- Ký hiệu Non-Resident File trong Hex Editor:
 - Giá trị **01** thể hiện tập tin là Non-Resident.



Bài tập (yêu cầu làm)

4. Thực hiện phân tích:

- Tài nguyên: [not-and-never.gz](https://not-and-never.github.io/)
 - Mô tả: Tìm thông tin có liên quan đến từ khóa "key" trong dữ liệu được cung cấp
- Gợi ý: MFT, mmls, dd, strings, foremost/scalpel

5. Thực hiện phân tích:

- Tài nguyên: corrupted-disk-image.vhd
- Mô tả: Bạn phát hiện một tệp ảnh đĩa (disk image) bị hỏng, được trích xuất từ laptop của một người bạn. Anh ta đã tải xuống một số tệp tin quan trọng, sau đó rời đi một lúc. Khi quay lại, anh ta phát hiện không thể truy cập ổ đĩa. Hệ thống tệp có vẻ đã bị **hỏng** hoặc **bị ghi đè**, nhưng ẩn sâu bên trong cấu trúc bị phá vỡ đó có thể chứa **những dữ liệu quan trọng**, giúp mở ra **manh mối** tiếp theo trong cuộc điều tra của bạn.
- Yêu cầu:

(1) Sau khi khôi phục ổ đĩa, người bạn của bạn đã tải xuống một tệp từ Google. Thời điểm chính xác anh ấy nhấn tải xuống là khi nào?

(2) Quá trình tải xuống tệp mất bao nhiêu giây để hoàn tất?

(3) Thư mục đầu tiên mà anh ấy di chuyển tệp đến là gì?

(4) Thư mục cuối cùng mà tệp bị di chuyển đến một cách đáng ngờ là gì?

(5) Thời điểm tệp bị xóa là khi nào?

Gợi ý: Manh mối nằm ở các tập tin \$MFT và \$J (công cụ MFTcmd thu thập tập tin csv, TimeLine Explorer để xem tập tin csv).

b) Khôi phục tập tin đã xóa trên NTFS

Bài tập (yêu cầu làm)

6. Thực hiện các bước sau:

- Trên máy tính/máy ảo windows thực hiện tải về hình ảnh (https://inseclab.uit.edu.vn/upload/2021/01/logo_inseclab-03.png).
- Thực hiện xóa file ảnh vừa tải, xóa trong Recycle Bin.
- Tạo một ảnh đĩa - định dạng Raw (dd) sau khi xóa file ảnh trên. Đặt tên Case Number, Evidence Number, Unique Description và Examiner (VD: March-0001, 01, InSecLab logo, N01). *Examiner đặt theo tên nhóm
- Tạo một thư mục điều tra dùng cho kịch bản này: KB03, chứa ảnh đĩa đã tạo.

Yêu cầu: Thực hiện điều tra, tìm ảnh đã bị xóa trên ổ đĩa bằng công cụ FTK Imager. Sử dụng tính năng khôi phục file ảnh đã bị xóa (tính năng Export Files), lưu trữ file này trong thư mục KB03\images. Sau đó, kiểm tra giá trị hash MD5 của file ảnh vừa được khôi phục với file gốc ban đầu.

B2.3. Linux File System

Linux hỗ trợ nhiều hệ thống tập tin khác nhau, mỗi loại có những ưu điểm riêng phù hợp với từng nhu cầu sử dụng. Một số hệ thống tập tin phổ biến trong Linux bao gồm: **Ext, XFS, Btrfs, ReiserFS, ZFS.**

Trong bài lab này, tập trung vào **Ext**, hệ thống tập tin hiện được nhiều bản phân phối Linux sử dụng mặc định, nhờ vào tính ổn định và khả năng mở rộng tốt.

B2.3.1. Hệ thống tập tin Ext

xtended Filesystem (Ext) là hệ thống tập tin đầu tiên được thiết kế dành riêng cho Linux. Trải qua bốn phiên bản phát triển, mỗi phiên bản đều mang đến những cải tiến đáng kể. Phiên bản đầu tiên của Ext được xây dựng dựa trên Minix filesystem nhưng không đáp ứng được nhu cầu của hệ điều hành hiện đại. Hiện tại, dù Ext4 là phiên bản mới nhất nhưng vẫn còn một số hạn chế và bị loại bỏ trên một số bản phân phối Linux.

Tuy nhiên, Ext vẫn là một hệ thống tập tin phổ biến, đặc biệt trong quá trình cài đặt một số hệ điều hành như CentOS 7, người dùng vẫn có thể gặp tùy chọn Ext4 trong bước phân vùng ổ cứng.

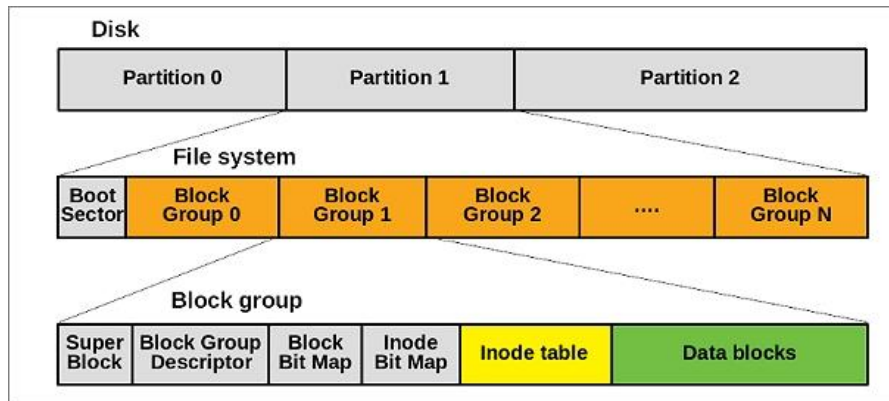
* Cấu trúc dữ liệu của Ext Filesystem:

Ext filesystem được tổ chức thành nhiều block và các block này được nhóm lại thành block group. Một hệ thống tập tin lớn có thể chứa hàng nghìn block group.

Dữ liệu của một file sẽ được lưu trong một block group duy nhất nhằm giảm số lần truy xuất trên đĩa, cải thiện hiệu suất khi đọc dữ liệu tuần tự.

Mỗi block group bao gồm:

- **Superblock:** Chứa thông tin quan trọng về toàn bộ hệ thống tập tin.
- **Descriptor table:** Bảng mô tả block group.
- **Block bitmap:** Theo dõi block nào đã được sử dụng.
- **Inode bitmap:** Theo dõi inode nào đã được cấp phát.
- **Inode table:** Lưu trữ thông tin về các tệp tin.
- **Data block:** Chứa dữ liệu thực tế của tệp tin.



(Nguồn: EaseUS)

B2.3.1. Các phiên bản của Ext Filesystem

a) Ext2

Ext2 (Second Extended Filesystem) là phiên bản kế thừa từ Ext đầu tiên, được thiết kế để cải thiện khả năng quản lý dung lượng lưu trữ và hiệu suất. Điểm đáng chú ý là Ext2 không hỗ trợ journaling, giúp giảm số lần ghi lên ổ đĩa, do đó phù hợp với các thiết bị lưu trữ ngoài như USB hoặc thẻ nhớ SD.

Tuy nhiên, Ext2 có những hạn chế về kích thước tệp và dung lượng tối đa của ổ đĩa. Trên các phiên bản Linux kernel cũ hơn 2.6.17, Ext2 chỉ hỗ trợ tối đa 2TB dung lượng.

b) Ext3

Ext3 được phát triển từ Ext2 nhưng bổ sung journaling, giúp cải thiện độ tin cậy của hệ thống và giảm thời gian kiểm tra lỗi sau khi mất điện hoặc tắt máy đột ngột. Ext3 xuất hiện từ Linux kernel 2.4.15 và có thể nâng cấp trực tiếp từ Ext2 mà không cần format lại ổ đĩa.

Các tính năng mới của Ext3 so với Ext2:

- **Journaling:** Giúp tăng độ tin cậy bằng cách ghi lại các thay đổi trước khi thực sự áp dụng lên hệ thống tập tin.
- **Tăng kích thước filesystem online:** Cho phép mở rộng dung lượng ổ đĩa mà không cần downtime.

- **Directory Indexing:** Sử dụng cấu trúc HTree để tăng hiệu suất truy xuất thư mục lớn.

Dù có những cải tiến, nhưng hiệu suất của Ext3 không thể so sánh với các hệ thống tập tin hiện đại như Ext4 hay XFS.

c) Ext4

Ext4 ban đầu được phát triển như một tập hợp các cải tiến mở rộng cho Ext3 nhằm tăng cường hiệu suất và khả năng mở rộng. Tuy nhiên, do lo ngại về độ tin cậy của những cải tiến này, một nhóm lập trình viên đã tách riêng Ext3 và phát triển thành một hệ thống tập tin độc lập mang tên Ext4.

Ext4 chính thức xuất hiện trong Linux kernel 2.6.19 và trở thành phiên bản ổn định từ kernel 2.6.28. Ngày 15/01/2010, Google thông báo sẽ nâng cấp hệ thống lưu trữ của họ từ Ext2 lên Ext4.

* Những cải tiến quan trọng của Ext4:

- **Dung lượng tối đa:** Hỗ trợ ổ đĩa lên đến 1 exbibyte (EiB) và tập tin có kích thước tối đa 16 tebibytes (TiB).
- **Extent-based storage:** Thay vì sử dụng block mapping như Ext2 và Ext3, Ext4 sử dụng "extent" – một nhóm block liên tục trên đĩa – giúp tăng hiệu suất và giảm phân mảnh.
- **Tương thích ngược:** Có thể gắn Ext2 và Ext3 vào hệ thống chạy Ext4 mà không gặp vấn đề.
- **Số lượng thư mục con không giới hạn:** Ext3 chỉ hỗ trợ tối đa 32.000 thư mục con trong một thư mục, trong khi Ext4 loại bỏ giới hạn này.
- **Journal checksumming:** Sử dụng checksum để bảo vệ dữ liệu trong journal, giúp tăng độ tin cậy.
- **Metadata checksumming:** Hỗ trợ từ Linux kernel 3.5, giúp bảo vệ dữ liệu quan trọng.

Bài tập (yêu cầu làm)

7. Thực hiện phân tích:

- Tài nguyên: *gone-gone-gone.img.tar.bz2*
- Mô tả: *Hình như các tập tin của chúng ta đã bị đánh cắp!*

Gợi ý: *extundelete, fsck.ext4, binwalk, openssl*

C. YÊU CẦU & ĐÁNH GIÁ

1. Yêu cầu

- Sinh viên tìm hiểu và thực hành theo hướng dẫn. Đăng ký nhóm cố định từ buổi 1.
- Sinh viên báo cáo kết quả thực hiện và nộp bài bằng **1 trong 2 hình thức:**

c) Hình thức 1 - Báo cáo chi tiết:

Báo cáo cụ thể quá trình thực hành (có ảnh minh họa các bước) và giải thích các vấn đề kèm theo. Trình bày trong file PDF theo mẫu có sẵn tại website môn học.

d) Hình thức 2 - Video trình bày chi tiết:

Quay lại quá trình thực hiện Lab của sinh viên kèm thuyết minh trực tiếp mô tả và giải thích quá trình thực hành. Upload lên **Youtube** và chèn link vào đầu báo cáo theo mẫu. **Lưu ý:** Không chia sẻ ở chế độ Public trên Youtube.

Đặt tên file báo cáo theo định dạng như mẫu:

[Mã lớp]-LabX_MSSV1 _MSSV2

Ví dụ: [NT101.N21.ATCL.1]-Lab1_20520000 _20520999.

- Nếu báo cáo có nhiều file, nén tất cả file vào file .ZIP với cùng tên file báo cáo.
- Nộp báo cáo trên theo thời gian đã thống nhất tại website môn học.

2. Đánh giá:

- Sinh viên hiểu và tự thực hiện được bài thực hành, đóng góp tích cực tại lớp.
- Báo cáo trình bày chi tiết, giải thích các bước thực hiện và chứng minh được do nhóm sinh viên thực hiện.
- Hoàn tất nội dung cơ bản và có thực hiện nội dung *mở rộng – cộng điểm* (với lớp ANTN).

Kết quả thực hành cũng được đánh giá bằng kiểm tra kết quả trực tiếp tại lớp vào cuối buổi thực hành hoặc vào buổi thực hành thứ 2.

Lưu ý: Bài sao chép, nộp trễ, “*gánh team*”, ... sẽ được xử lý tùy mức độ.

HẾT

Chúc các bạn hoàn thành tốt!